Credit Card Approval Prediction

Project Description:

The Credit Card Approval Prediction system is a Flask-based machine learning web application designed to support automated screening of credit card applications. It collects applicant profile and financial information, stores applicant and credit-related records in a database, sends prepared applicant data to a machine learning prediction service, and returns an approval or rejection prediction with a corresponding risk category.

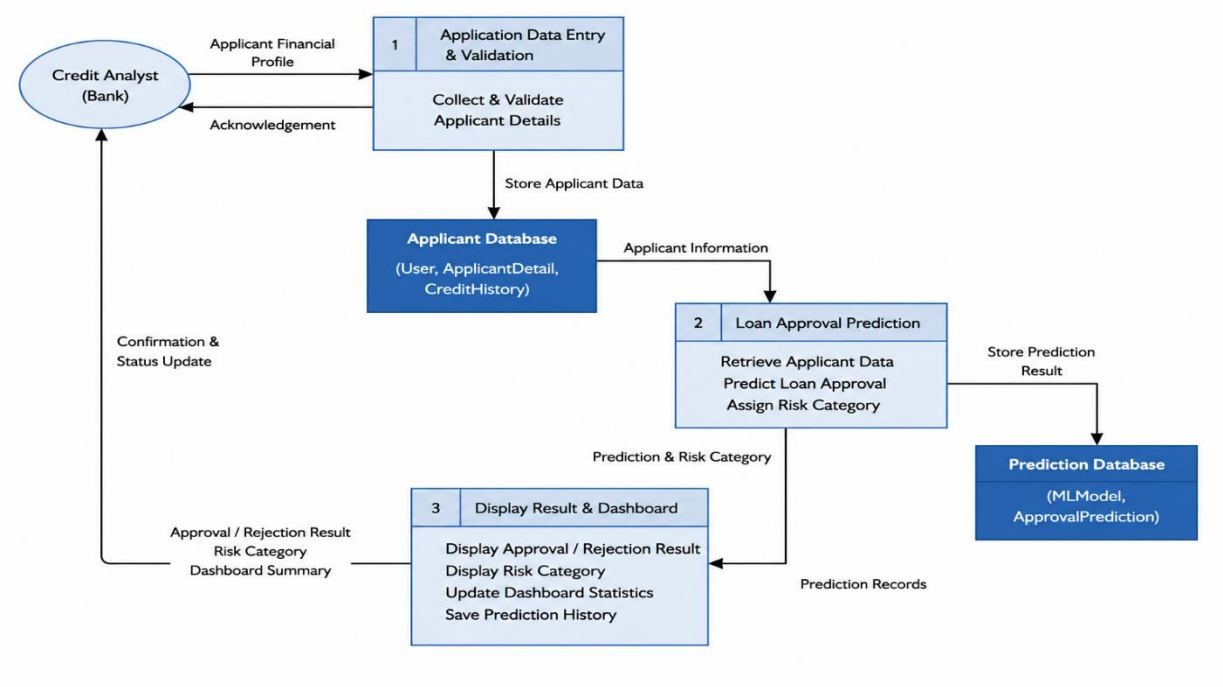
The system uses Flask for the web application and Flask-SQLAlchemy for ORM-based database access. Its database structure includes User, ApplicantDetail, CreditHistory, MLModel, and ApprovalPrediction. The application integrates a CreditPredictor from src.prediction and maintains model metadata for an XGBoost Classifier and an Ensemble Classifier (Random Forest + XGBoost). The dashboard presents total applicants, total predictions, approval and rejection rates, active model information, and the five most recent predictions

Scenarios

Scenario : Automated Credit Card Application Screening

A bank credit analyst enters a new applicant's financial and profile information, including income type, employment duration, and credit-related information, into the web application. The system stores the applicant details and sends the prepared data to the machine learning prediction module. The model returns an approval or rejection prediction, helping the analyst screen applications efficiently.

Technical Architecture:



Description:

The Data Flow Diagram represents the workflow of the **Credit Card Approval Prediction System**.
The **Credit Analyst** enters the applicant's financial profile into the application.
The system collects, validates, and stores the applicant details in the **Applicant Database**.
The stored applicant information is sent to the **Loan Approval Prediction** process.
The machine learning model predicts the **approval or rejection result** and assigns a **risk category**.
The prediction result is stored in the **Prediction Database** for future reference.
The system displays the prediction result, risk category, and updates the dashboard statistics.
Finally, the **Credit Analyst** receives the approval/rejection result and dashboard summary.

Pre-requisites:

- **Python 3.x** installed on the system
- **pip** (Python Package Manager)
- **Flask** framework
- **Flask-SQLAlchemy** for database management
- Required **Machine Learning libraries** for the prediction model
- **VS Code** or any Python IDE/code editor
- Basic knowledge of **Python, Flask, HTML, CSS, and JavaScript**
- Required project files: `app.py`, `config.py`, `src/prediction.py`, `requirements.txt`, and the `templates` folder
- Required trained **Machine Learning model files**
- A modern **web browser** to access and test the application

Database / ER Model Description

Entity	Purpose	Relationship
User	Stores application user information such as name, email, password, and role.	One User can have many ApplicantDetail records.
ApplicantDetail	Stores applicant profile and employment information.	Belongs to User; has many CreditHistory records and one ApprovalPrediction.
CreditHistory	Stores monthly credit repayment and overdue information.	Belongs to ApplicantDetail.
MLModel	Stores machine learning model metadata.	One MLModel can be linked to many ApprovalPrediction records.
ApprovalPrediction	Stores approval result, risk category, model reference, applicant reference, and prediction date.	Belongs to one ApplicantDetail and one MLModel.

Project Workflow:

Milestone 1: Model Selection and Architecture

- Identify credit card approval prediction requirements.
- Define the architecture connecting the web interface, Flask backend, database, and prediction module.
- Define relational database models and relationships.
- Set up the Python development environment and dependencies.

.

Milestone 2: Core Functionalities Development

- Develop applicant data collection and storage.
- Integrate CreditPredictor and approval/rejection prediction.
- Assign risk categories and store prediction results.
- Develop dashboard statistics and recent-prediction retrieval.

Milestone 3: App.py Development

- Configure Flask, SQLAlchemy, logging, and application configuration.
- Define the five database models.
- Implement database initialization and default data seeding.
- Implement GET /, GET /predict, and POST /predict routes.

Milestone 4: Frontend Development

- Develop home.html for the dashboard.
- Develop index.html for applicant data entry.
- Develop result.html for prediction outcome and risk category.

Milestone 5: Deployment

- Install dependencies and verify project files.
- Initialize the database and run locally.
- Verify submission, prediction, storage, and dashboard updates.
- Use Waitress through the existing production startup path.

Milestone 6: Conclusion

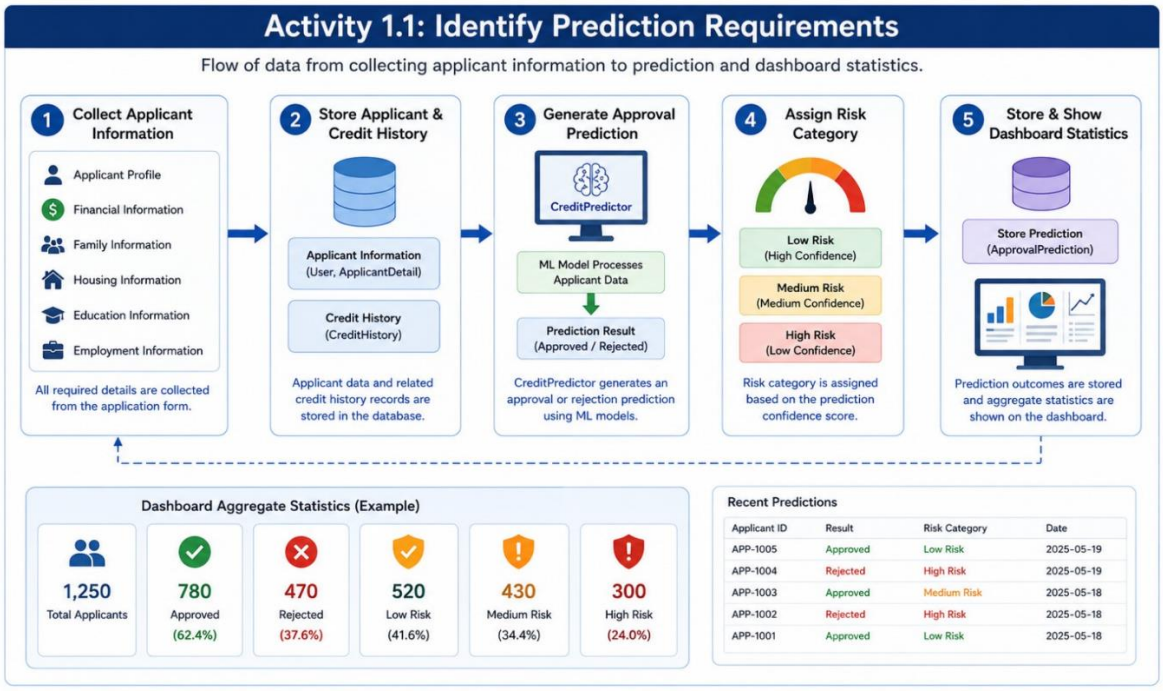
- Summarize the completed prediction workflow, database integration, dashboard, and deployment behavior.

Milestone 1: Model Selection and Architecture

This milestone defines the system architecture and prediction workflow. The application separates web request handling, database persistence, configuration, prediction logic, and HTML templates.

Activity 1.1: Identify Prediction Requirements

- Collect applicant profile, financial, family, housing, education, and employment-related information required by the application.
- Store applicant information and related credit history records.
- Generate an approval or rejection prediction using CreditPredictor.
- Assign a Low, Medium, or High risk category from prediction confidence.
- Store prediction outcomes and show aggregate statistics on the dashboard.



Activity 1.2: Define the Application Architecture

Here are the things that needed to research to select a best model for the project:

- **Presentation Layer:** home.html, index.html, and result.html.
- **Application Layer:** app.py handles routing, form processing, statistics, database operations, and prediction integration.
- **Prediction Layer:** src.prediction exposes CreditPredictor and predict_approval(applicant_data_dict).
- **Data Layer:** Flask-SQLAlchemy persists users, applicants, credit history, model metadata, and predictions.
- **Configuration Layer:** config.get_config() provides application configuration.

Activity 1.3: Define the Architecture of the Application

- The relational database structure is designed to connect all the important components of the application in an organized manner.
- **Users** are linked to their respective **applicant records**, while each applicant can have associated **credit history information** and a stored **approval prediction result**.
- The prediction records are also connected to the corresponding **machine learning model metadata**, which stores information about the model used for prediction.
- This structured relationship allows the system to efficiently store, retrieve, and manage applicant details, credit history, machine learning model information, and prediction outcomes.

Activity 1.4: Set Up the Development Environment

- **Install Python and pip:** Install Python on the system and verify that pip, the Python package manager, is available for installing the required project libraries.
- **Open the Project in VS Code:** Open the complete Credit Card Approval Prediction project folder in VS Code or another suitable development environment to access and manage all project files.
- **Create a Virtual Environment:** Create and activate a Python virtual environment if required. This helps keep the project dependencies separate from other Python projects installed on the system.
- **Install Required Dependencies:** Install all necessary Python libraries and frameworks using the requirements.txt file to ensure that the Flask application and machine learning prediction module work correctly**.**
- **Verify the Project Files:** Check that important files and folders such as app.py, config.py, src/prediction.py, the templates folder, and the required machine learning model files are available in the correct project structure.
- **Run the Application:** Execute app.py to initialize the database, create the required tables and default records, and start the Flask web application for testing and prediction.

Milestone 2: Core Functionalities Development

Activity 2.1: Applicant Data Collection and Storage

- Collect applicant personal, financial, employment, education, family, and housing details.
- Read and process the submitted form data using Flask.
- Calculate required values such as applicant age and employment duration.
- Store the processed information in the ApplicantDetail table.
- Create a related CreditHistory record for the applicant.

Activity 2.2: Machine Learning Prediction Integration

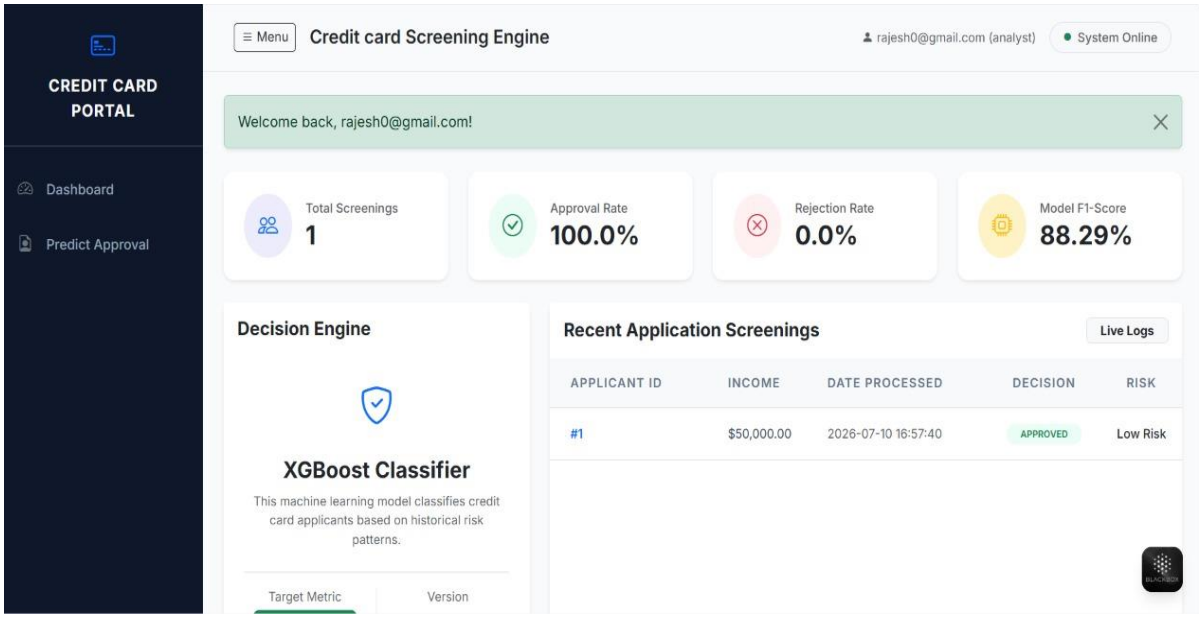
- Load the CreditPredictor from src.prediction using get_predictor().
- Prepare the applicant information as input for the prediction model.
- Pass the applicant data to the predict_approval() method.
- Generate an Approved or Rejected prediction.
- Use the prediction confidence for further risk categorization.

Activity 2.3: Risk Categorization and Prediction Storage

- Store the final prediction as **Approved** or **Rejected**.
- Assign **Low Risk** when confidence is greater than or equal to 0.80.
- Assign **Medium Risk** when confidence is greater than or equal to 0.50.
- Assign **High Risk** when confidence is below 0.50.
- Store the final result in the ApprovalPrediction table.

Activity 2.4: Dashboard Statistics

- Count the total number of applicants.
- Count the total number of predictions.
- Calculate the approval and rejection rates.
- Display the active machine learning model information.
- Retrieve and display the five most recent predictions.


CREDIT CARD PORTAL
Dashboard
Predict Approval
Menu
Credit card Screening Engine
rajesh0@gmail.com (analyst)
System Online
Welcome back, rajesh0@gmail.com!
Total Screenings
1
Approval Rate
100.0%
Rejection Rate
0.0%
Model F1-Score
88.29%
Decision Engine
XGBoost Classifier
This machine learning model classifies credit card applicants based on historical risk patterns.
Target Metric
Version
Recent Application Screenings
Live Logs
APPLICANT ID
INCOME
DATE PROCESSED
DECISION
RISK
#1
$50,000.00
2026-07-10 16:57:40
APPROVED
Low Risk

Milestone 3: App.py Development

The app.py file acts as the main controller and central module of the Credit Card Approval Prediction System. It connects the Flask web application, database, and machine learning prediction module. It is responsible for defining application routes, processing applicant form data, storing applicant and credit history records, generating approval predictions, assigning risk categories, and saving prediction results.

Activity 3.1: Application Setup

The application setup includes the following tasks:

- Create and configure the Flask application.
- Load application settings using config.get_config().
- Initialize Flask-SQLAlchemy for database operations.
- Configure informational logging for monitoring application activity.

These steps establish the basic environment required to run the web application and connect its different components.

Credit Card Approval Prediction/

```
|
├── app.py
├── config.py
├── requirements.txt
|
├── src/
|   └── prediction.py
|
├── templates/
|   ├── home.html
|   ├── index.html
|   └── result.html
|
└── Database
    ├── User
    ├── ApplicantDetail
```

- CreditHistory
- MLModel
- ApprovalPrediction

Matter: User Database

The Users table stores the details of application users. It contains fields such as UserID, Name, Email, Password, and Role. This table is used to manage users such as analysts who access and operate the credit card screening application.

```
-- ============================================================
-- TABLE 1: Users
-- ============================================================
CREATE TABLE IF NOT EXISTS Users (
    UserID      INTEGER PRIMARY KEY AUTOINCREMENT,
    Name        VARCHAR(100)  NOT NULL,
    Email       VARCHAR(150)  NOT NULL UNIQUE,
    Password    VARCHAR(255)  NOT NULL,
    Role        VARCHAR(50)   NOT NULL DEFAULT 'analyst'
);

SELECT * FROM Users;
-- ============================================================
-- TABLE 2: Applicant_Details
-- (Expanded to store ALL form fields submitted during credit screening)
-- ============================================================
```

Matter: Applicant Details Database

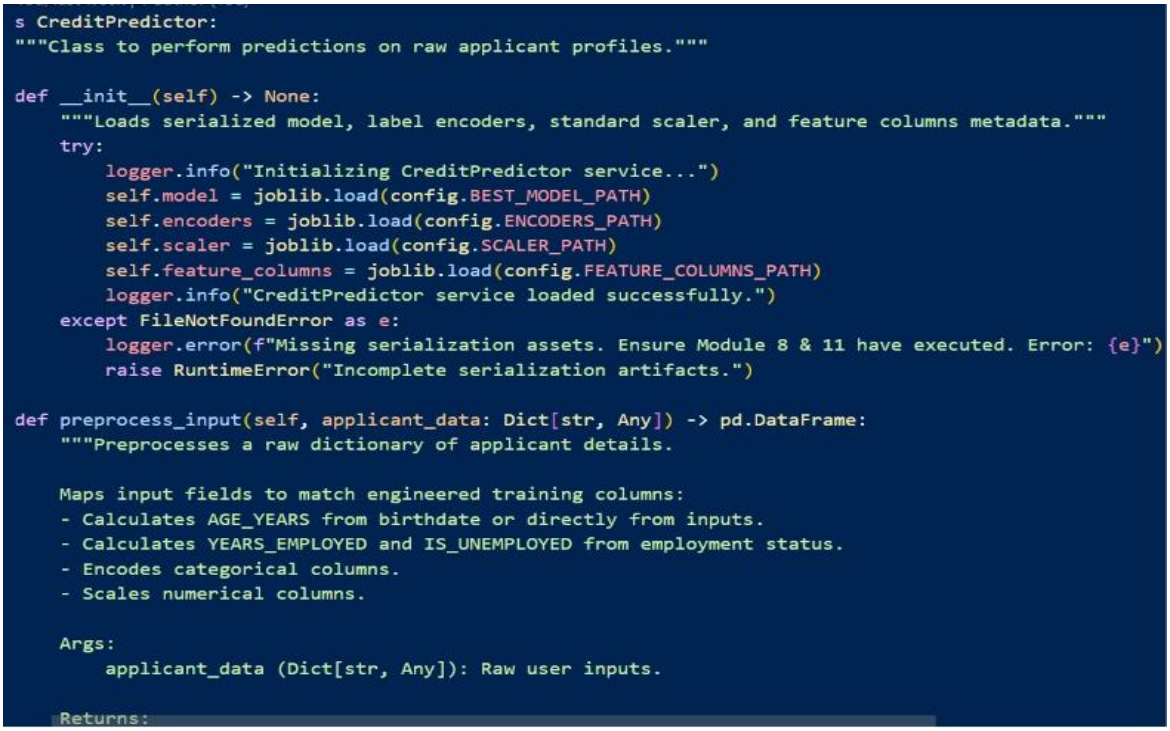
The Applicant_Details table stores detailed applicant information required for credit screening. It includes fields such as gender, car and property ownership, income, education, family status, housing type, employment duration, occupation, and family member count.

```
CREATE TABLE IF NOT EXISTS Applicant_Details (
    ApplicantID         INTEGER PRIMARY KEY AUTOINCREMENT,
    UserID              INTEGER         NOT NULL,
    Gender              VARCHAR(10)     NOT NULL DEFAULT 'Unknown',
    OwnCar              VARCHAR(5)      NOT NULL DEFAULT 'N',
    OwnRealty           VARCHAR(5)      NOT NULL DEFAULT 'N',
    ChildrenCount       INTEGER         NOT NULL DEFAULT 0,
    IncomeTotal         FLOAT           NOT NULL DEFAULT 0.0,
    IncomeType          VARCHAR(50)     NOT NULL,
    EducationType       VARCHAR(100)    NOT NULL,
    FamilyStatus        VARCHAR(50)     NOT NULL,
    HousingType         VARCHAR(50)     NOT NULL,
    DaysBirth           INTEGER         NOT NULL DEFAULT 0,
    EmploymentDays      INTEGER         NOT NULL,
    WorkPhone           INTEGER         NOT NULL DEFAULT 0,
    Phone               INTEGER         NOT NULL DEFAULT 0,
    EmailFlag           INTEGER         NOT NULL DEFAULT 0,
    OccupationType      VARCHAR(100)    NOT NULL DEFAULT 'Unknown',
    FamilyMembersCount  FLOAT           NOT NULL DEFAULT 1.0,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

▷Run | +Tab | JSON
SELECT * FROM Applicant_Details;
```

Matter: Credit Prediction and Preprocessing Module

The CreditPredictor class loads the trained model, label encoders, standard scaler, and feature column information. It preprocesses raw applicant details by calculating required features, encoding categorical data, and scaling numerical values before generating the credit approval prediction.



```
s CreditPredictor:
"""Class to perform predictions on raw applicant profiles."""

def __init__(self) -> None:
    """Loads serialized model, label encoders, standard scaler, and feature columns metadata."""
    try:
        logger.info("Initializing CreditPredictor service...")
        self.model = joblib.load(config.BEST_MODEL_PATH)
        self.encoders = joblib.load(config.ENCODERS_PATH)
        self.scaler = joblib.load(config.SCALER_PATH)
        self.feature_columns = joblib.load(config.FEATURE_COLUMNS_PATH)
        logger.info("CreditPredictor service loaded successfully.")
    except FileNotFoundError as e:
        logger.error(f"Missing serialization assets. Ensure Module 8 & 11 have executed. Error: {e}")
        raise RuntimeError("Incomplete serialization artifacts.")

def preprocess_input(self, applicant_data: Dict[str, Any]) -> pd.DataFrame:
    """Preprocesses a raw dictionary of applicant details.

    Maps input fields to match engineered training columns:
    - Calculates AGE_YEARS from birthdate or directly from inputs.
    - Calculates YEARS_EMPLOYED and IS_UNEMPLOYED from employment status.
    - Encodes categorical columns.
    - Scales numerical columns.

    Args:
        applicant_data (Dict[str, Any]): Raw user inputs.

    Returns:
```

Matter: Model Evaluation Module

The ModelEvaluator class is used to evaluate and compare trained machine learning models. It calculates evaluation metrics, generates confusion matrices, and prepares performance reports to compare the performance of different models.

```python
class ModelEvaluator:
    """Class to evaluate and compare trained machine learning models."""

    def __init__(self) -> None:
        """Initialize evaluator and ensure visual output path exists."""
        self.image_dir = config.BASE_DIR / "static" / "images"
        self.image_dir.mkdir(parents=True, exist_ok=True)

        # Set evaluation plot parameters
        sns.set_theme(style="whitegrid", context="talk")

    def evaluate_all(
        self,
        models: Dict[str, Any],
        X_test: np.ndarray,
        y_test: np.ndarray
    ) -> pd.DataFrame:
        """Computes evaluation metrics, confusion matrices, and prints reports.

        Args:
            models (Dict[str, Any]): Dictionary of trained model objects.
            X_test (np.ndarray): Test features.
            y_test (np.ndarray): Test targets.

        Returns:
            pd.DataFrame: Table comparing the metrics of all models.
        """
        logger.info("Starting model evaluation pipeline...")
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for CaptionAI. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated captions, hashtags, and CTAs based on user interactions.

Activity 4.1: Dashboard Page — home.html

Displays total applicants, total predictions, approval and rejection rates, active model metadata, and the five most recent predictions.

- Displays key statistics such as **total screenings, approval rate, rejection rate, and model F1-score**.
- Shows the active machine learning model, such as the **XGBoost Classifier**.
- Displays recent application screenings with **Applicant ID, Income, Date Processed, Decision, and Risk Category**.
- Provides navigation options to access the **Dashboard** and **Predict Approval** pages.
- Gives the analyst a clear and organized overview of the system's credit card screening activities.

Activity 4.2: Prediction Form — index.html

Provides the interface for entering applicant information and submitting it to POST /predict.

Handle Form Submission Asynchronously:

- Attach a submit event listener to the generator form that prevents default form submission and sends the data as a JSON POST request to /generate via the Fetch API.
- During loading, disable the submit button and show an animated CSS loader in place of the button text and arrow icon.

Activity 4.3: Prediction Result — result.html

Displays the generated approval outcome and corresponding risk category.

- Displays the final **credit card approval prediction** as **Approved** or **Rejected**.
- Shows the corresponding **risk category**, such as **Low Risk, Medium Risk, or High Risk**.
- Presents the prediction outcome in a clear and easy-to-understand format.
- Helps the credit analyst quickly review the result generated by the machine learning model.
- Provides the final output after the applicant details are processed by the **CreditPredictor**.

Milestone 5: Deployment and Verification

This milestone focuses on preparing the project environment, running the application, and verifying that all major components work correctly.

Activity 5.1: Prepare the Application

- Open the Credit Card Approval Prediction project folder in VS Code and launch the terminal.
- Create and activate a Python virtual environment if required to manage project dependencies separately.
- Install all required Python libraries using the requirements.txt file.
- Verify that important files such as app.py, config.py, and src/prediction.py are available.
- Confirm that the templates folder and required trained machine learning model files are present.

Activity 5.2: Run the Application Locally

- Open the terminal in the project root directory and install dependencies using pip install -r requirements.txt.
- Start the Flask application by running the python app.py command.
- Wait for the database initialization process and Flask server to start successfully.
- Open http://127.0.0.1:5000 in a web browser to access the application.
- Enter applicant details in the prediction form and submit them for prediction.
- Verify the approval/rejection result, risk category, stored prediction record, and updated dashboard statistics.

Activity 5.3: Production Startup

- When app.py is executed, the initialize_database() function prepares the required database tables and initial records.
- The application checks the FLASK_ENV environment setting to determine the startup mode.
- In the development environment, the application runs using the Flask development server.
- In the production environment, the application uses the Waitress WSGI server.
- The application is configured to run on 0.0.0.0:5000.

Milestone 5: Deployment and Verification

This milestone focuses on preparing the project environment, running the application, and verifying that all major components work correctly.

Activity 5.1: Prepare the Application

1. Open the Credit Card Approval Prediction project folder in VS Code and launch the terminal.
2. Create and activate a Python virtual environment if required.
3. Install all required Python libraries using the `requirements.txt` file.
4. Verify that important files such as `app.py`, `config.py`, and `src/prediction.py` are available.
5. Confirm that the `templates` folder and required trained machine learning model files are present.

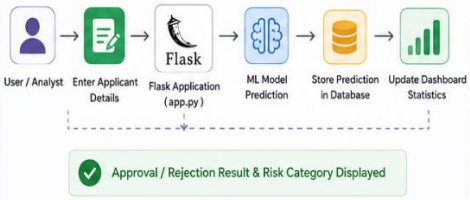
Activity 5.2: Run the Application Locally

1. Open the terminal in the project root directory and install dependencies using `pip install -r requirements.txt`.
2. Start the Flask application by running the `python app.py` command.
3. Wait for the database initialization process and Flask server to start successfully.
4. Open `http://127.0.0.1:5000` in a web browser to access the application.
5. Enter applicant details in the prediction form and submit them for prediction.
6. Verify the approval/rejection result, risk category, stored prediction record, and updated dashboard statistics.

Activity 5.3: Production Startup

1. When app.py is executed, the `initialize_database()` function prepares the required database tables and initial records.
2. The application checks the `FLASK_ENV` environment setting to determine the startup mode.
3. In the development environment, the application runs using the Flask development server.
4. In the production environment, the application uses the Waitress WSGI server.
5. The application is configured to run on `0.0.0.0:5000`.

Application Flow



Web Pages

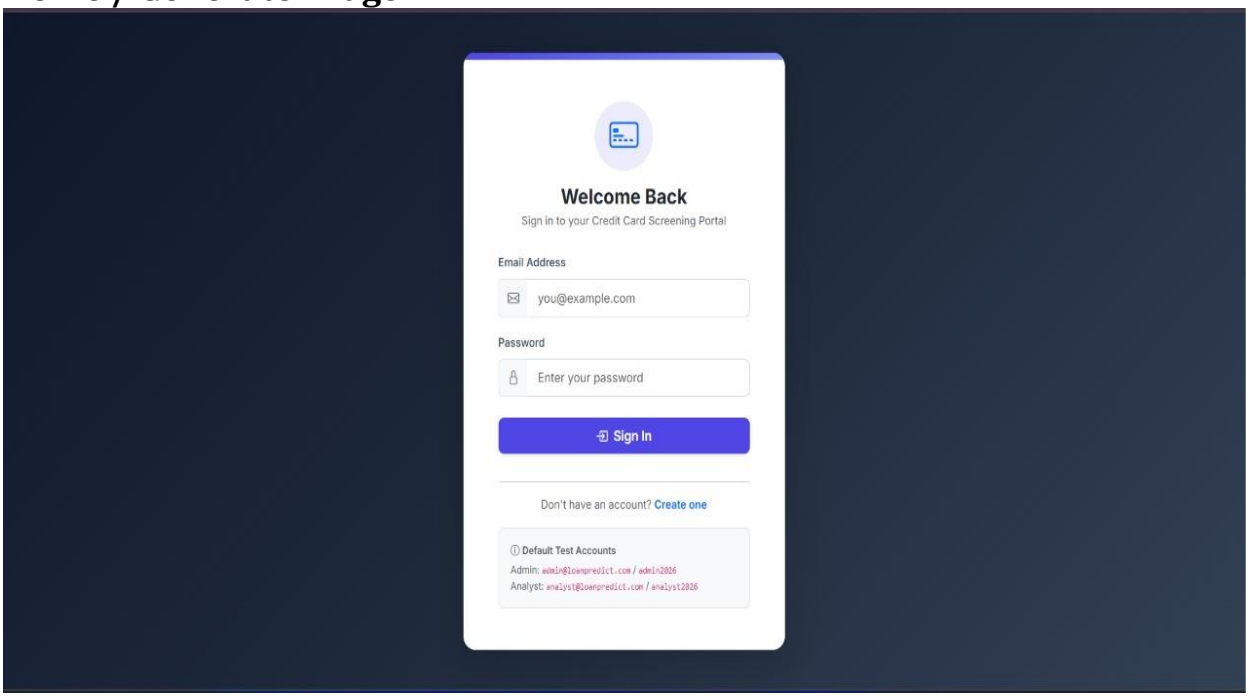
View total applicants, predictions, approval & rejection rates, active model, and recent predictions.

Enter applicant information required for credit card approval prediction.

Displays approval/rejection result, risk category, and application details.

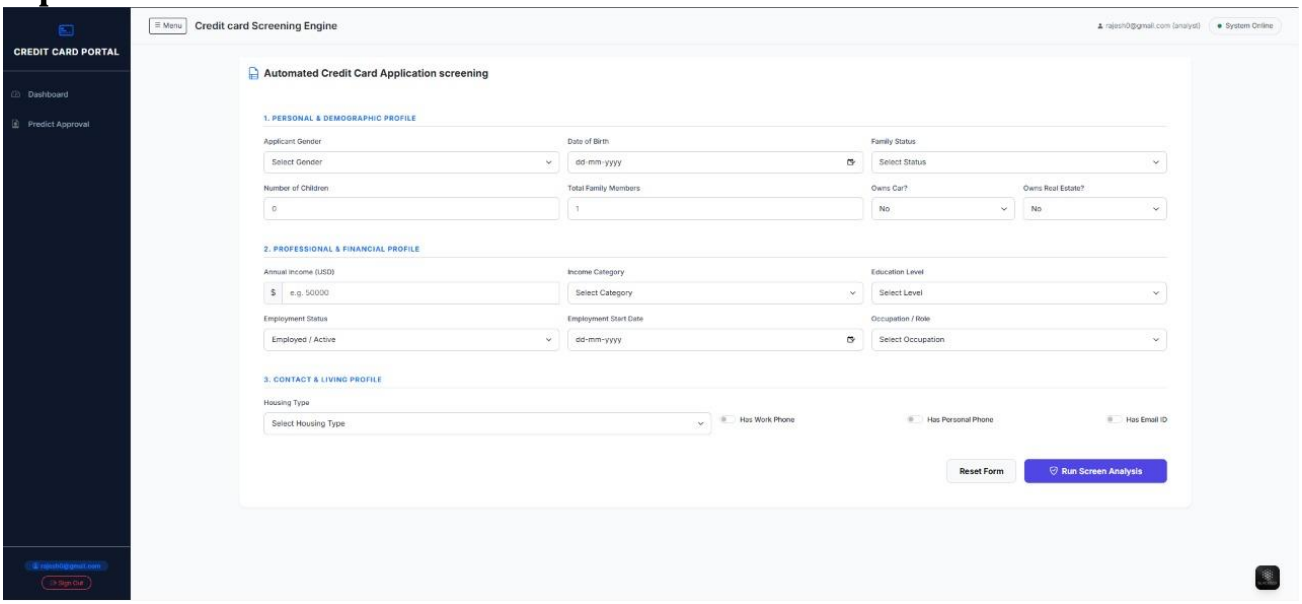
Exploring the Website's Web Pages:

Home / Generator Page:



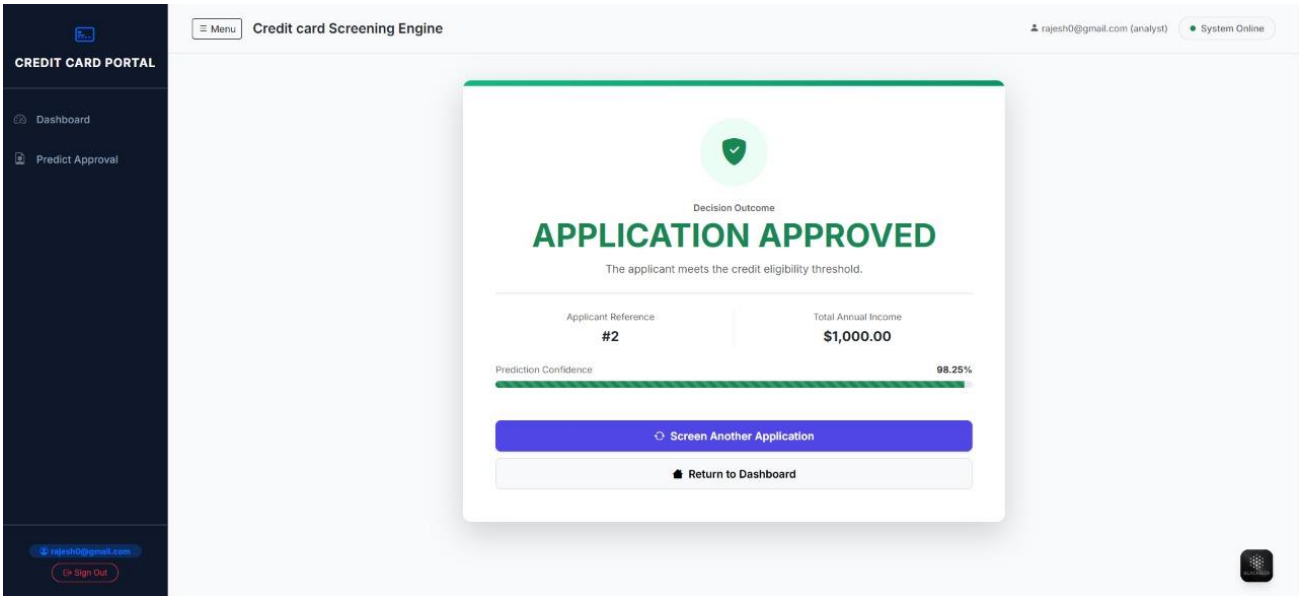
Description: The Dashboard serves as the main landing page of the **Credit Card Approval Prediction System**. It provides a clear overview of the application by displaying key statistics such as **total screenings, approval rate, rejection rate, and model performance score**. The page also displays information about the active machine learning model, such as the **XGBoost Classifier**, along with recent application screenings. A sidebar navigation menu allows the credit analyst to easily access the **Dashboard** and **Predict Approval** sections.

Input Section:



Description: The Prediction Input Page contains the main application form used by the credit analyst to enter applicant information. Users provide the required **personal, financial, employment, education, family, housing, and credit-related details**. After completing the form, the analyst submits the information for prediction. The Flask backend processes and stores the applicant data, prepares the required input features, and sends them to the CreditPredictor for machine learning prediction.

Results Section:



Description: Description: The Prediction Result Page is displayed after the applicant information has been successfully processed by the machine learning model. It clearly presents the final **Approved or Rejected** outcome along with the corresponding **Low Risk, Medium Risk, or High Risk** category. The prediction result is stored in the database for future reference, and the dashboard statistics and recent prediction records are updated to reflect the latest application screening.

Conclusion:

The **Credit Card Approval Prediction System** is a Flask-based machine learning web application designed to automate and simplify the initial screening of credit card applications. The system collects applicant and credit-related information, processes the data using the CreditPredictor, and generates an **Approved or Rejected** prediction along with a corresponding **Low, Medium, or High Risk** category.

The application uses **Flask-SQLAlchemy** to organize and manage data through the User, ApplicantDetail, CreditHistory, MLModel, and ApprovalPrediction models. It also provides a dashboard that displays important information such as **total screenings, approval and rejection rates, active model details, and recent prediction records**.

Overall, the project successfully combines **machine learning, Flask web development, database management, prediction storage, risk categorization, and dashboard reporting** into a structured application. Its modular architecture makes the system easier to maintain and provides a strong foundation for reliable operation in both development and production environments.